SRM Institute of Science and Technology

Cognitive Multi-Robot Search and Rescue

Through Collaborative Perception and Reasoning

Project Report — Design Assignment 2

Course:	【Course Code / Name】
Guide:	【Faculty Guide Name】
Submitted by:	Team of Four (listed overleaf)
Date:	【Submission Date】

Project Team

Name	Registration Number
Aadavan Elavalagan	23BRS1268
K S Vinoth	23BRS1209
Shyam Sundar	23BRS1192
Madhavan A Ramanujan	23BRS1227

Abstract

Search and rescue (SAR) operations in disaster-hit environments demand rapid, thorough coverage of large and hazardous areas — conditions under which a single robot is fundamentally limited. This report presents the design, implementation, and evaluation of a cognitive multi-robot SAR system in which a team of three simulated robots collaboratively maps a collapsed-structure environment, detects and shares victim discoveries through a common semantic memory, and divides the search workload through a learned task-allocation policy. The system was developed on ROS 2 (Jazzy) with the Gazebo Harmonic simulator. Each robot performs lidar-based SLAM; per-robot maps are fused into a single global grid; a camera-based detector projects victim sightings into the map frame; and a shared semantic-map node makes every discovery available to the whole team. Two interchangeable task allocators were built: a greedy nearest-frontier baseline and a multi-agent reinforcement learning policy (MAPPO, centralized training with decentralized execution) trained on a lightweight grid abstraction of the world, with rewards for exploration and victim discovery and penalties for redundant coverage and elapsed time. On held-out test maps, the learned policy matched the hand-crafted baseline in map coverage (85% vs 86%) and detected victims faster on average (43 vs 49 steps), while finding fewer victims per mission (3.25 vs 4.25 of 5) at the evaluated training budget of approximately 148,000 steps. The results establish a complete, tested, and reproducible system in which the remaining performance gap is attributable to training budget rather than design, and define a clear path — longer training, simulator-level benchmarking, and semantic ablation — toward the final deliverable.

Keywords: multi-robot systems; search and rescue; SLAM; semantic mapping; multi-agent reinforcement learning; MAPPO; ROS 2; Gazebo

Table of Contents

Abstract I

1. Introduction 2

2. Related Study 2

2.1 Urban search and rescue robotics 3

2.2 Multi-robot exploration and mapping 3

2.3 Multi-agent reinforcement learning 3

2.4 Tools and platforms 3

3. Proposed Work 4

3.1 System overview and architecture 4

3.2 Simulation environment and world generation 4

3.3 Collaborative mapping (O1) 4

3.4 Semantic perception and shared memory (O2, O3) 5

3.5 Task allocation: baseline and learned policy (O4) 5

3.6 Navigation and execution 5

3.7 Evaluation harness and metrics (O5) 6

4. Results and Analysis 6

4.1 Implementation verification 6

4.2 Training dynamics 6

4.3 Benchmark against baselines 7

4.4 Analysis 7

5. Conclusion and Future Work 8

References 8

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Introduction

Urban search and rescue remains one of the most demanding applications of robotics. After earthquakes, building collapses, or industrial accidents, response teams face structurally unstable spaces, limited visibility, and severe time pressure: the probability of finding survivors falls sharply with every passing hour. Ground robots are a natural aid in this setting because they can enter spaces that are unsafe for humans, but deploying only a single robot introduces a fundamental bottleneck — one platform can cover only so much of a large, cluttered environment before the mission window closes.

Introducing multiple robots multiplies coverage capacity, but it also introduces a coordination problem that grows with team size. Without deliberate coordination, robots cluster in the same partially explored areas, repeatedly re-observe the same space, and waste the very time budget the team was meant to save. Moreover, the raw occupancy maps that typical multi-robot systems merge answer only the question "what does the world look like" — not the question that actually matters for search: "where should each robot go next, given what has already been found?"

This project addresses that gap. Following our DA1 proposal, we built a multi-robot SAR system in which robots (a) jointly construct a global map, (b) enrich it with semantic information — located victims — held in a shared memory accessible to every robot, and (c) use that shared state to divide the search through a task-allocation policy that is learned rather than hand-coded. The concrete objectives, carried over from the DA1 abstract, are: (O1) collaborative SLAM-based mapping across robots; (O2) semantic scene understanding with victim detection; (O3) a shared memory layer queryable by all robots; (O4) a multi-agent reinforcement learning (MARL) policy for task allocation, trained as MAPPO under centralized training with decentralized execution (CTDE); and (O5) a benchmark of the learned policy against a rule-based allocation baseline.

The contributions of this phase of work are: a complete, modular ROS 2 implementation of the proposed architecture in eleven packages; a procedural world generator that guarantees the simulation environment matches the training environment; a custom multi-robot map-fusion node; a shared semantic-memory service with deduplication and stable identification; two interchangeable allocators with identical interfaces; a training stack (environment, MAPPO implementation, evaluation scripts) that trains on CPU in minutes; a contract test suite that pins the observation interface between training and deployment; and an initial benchmark with results reported in Section 4. The remainder of the report is organized as follows: Section 2 reviews related work; Section 3 details the proposed system; Section 4 reports results and analysis; Section 5 concludes with future work.

2. Related Study

2.1 Urban search and rescue robotics

Robot-assisted USAR has been studied for over two decades. Casper and Murphy [1] documented human–robot interaction during the robot-assisted response at the World Trade Center site and identified the core demands that still shape the field: rapid wide-area search, victim identification, and operation in confined, debris-strewn spaces. Their analysis makes explicit why multi-robot coverage matters — victim survival probability decays with time — and motivates the time-based metrics (time-to-detection, mission duration) adopted in this project.

2.2 Multi-robot exploration and mapping

Frontier-based exploration, introduced by Yamauchi [2], remains the dominant strategy for autonomous exploration: robots navigate to the boundaries between known free space and unknown space. Our rule-based allocator is a region-partitioned variant of this classic approach, and our learned allocator explicitly takes frontier information as input. The underlying mapping formulation follows the occupancy-grid and probabilistic-SLAM treatment of Thrun, Burgard, and Fox [3]; in implementation we use the open-source slam_toolbox stack on ROS 2 per robot, with a custom fusion node that stitches per-robot grids using known initial poses.

2.3 Multi-agent reinforcement learning

Cooperative MARL has converged on the CTDE paradigm: during training, a centralized component observes global state, while at execution each agent acts from local observations. MADDPG [4] established the pattern for continuous control; QMIX [5] factorizes a joint value function under monotonicity constraints; and COMA [6] introduces counterfactual baselines for credit assignment. Yu et al. [7] showed that a multi-agent variant of Proximal Policy Optimization — MAPPO, with a centralized value function and shared actor — matches or exceeds these specialized methods on standard benchmarks while being substantially simpler. We adopt MAPPO on that basis, with hyperparameters following the original paper. The underlying single-agent objective is PPO [8].

2.4 Tools and platforms

The system is built on ROS 2 Jazzy [9], chosen for its long-term-support status and multi-robot naming conventions, with the Gazebo Harmonic simulator [10] providing physics and sensor simulation. Victim detection follows the object-detection line initiated by YOLO [11], with the Ultralytics implementation [12]; because the simulation renders victims with a distinctive visual signature, a tuned HSV-blob detector is also provided as a dependency-

free fallback. Navigation uses the Nav2 stack [13] with the Smac 2D A* planner, matching the A*-based navigation stated in the DA1 abstract.

3. Proposed Work

3.1 System overview and architecture

Figure 1 shows the overall architecture. It is organized in three tiers. The per-robot tier is replicated for each of the three robots and contains everything that must be namespaced per platform: SLAM, perception, and navigation. The shared-state tier holds the globally consistent world model — the fused occupancy grid, the frontier/region decomposition, and the semantic map of located victims — each exposed as a ROS 2 topic or service that any robot can consume. The decision tier contains the two interchangeable task allocators and the evaluation harness. All interfaces are defined in a frozen contract document; the allocators publish identical goal messages, which is precisely what enables the controlled comparison in Section 4.

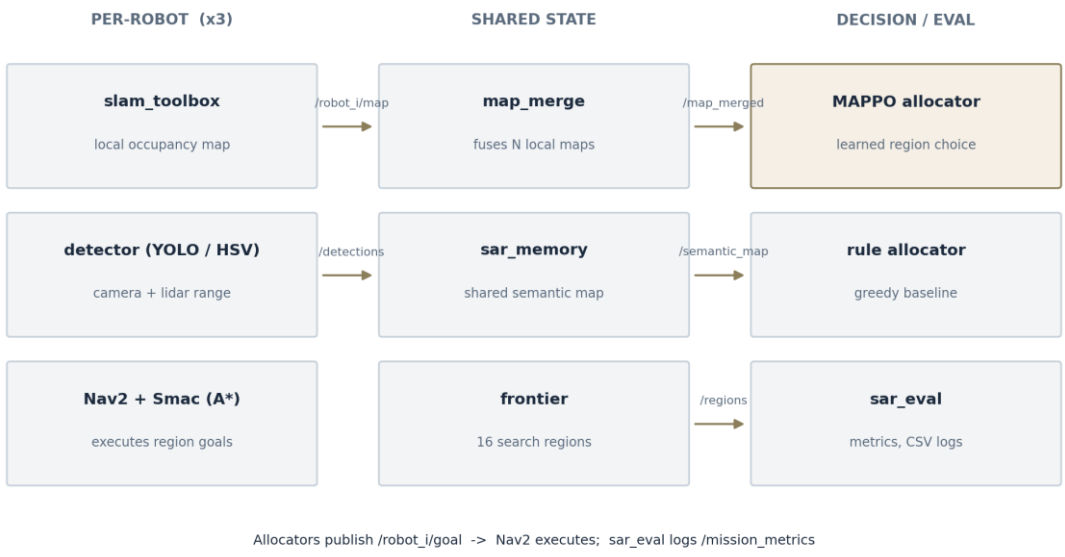


Figure 1: System architecture — per-robot tier (x3), shared state, and decision/evaluation tier, with the ROS 2 topics that connect them.

3.2 Simulation environment and world generation

The mission world is a 32 m × 32 m indoor collapsed-structure layout: room-and-corridor walls, five victim models with a bright visual signature for detection, and hazard props. Crucially, the world is not hand-authored: a generator script uses the same procedural maze code as the training environment, placing walls, robot spawn points, and victims from a single seed, and emits both the Gazebo world file and a ground-truth file listing exact victim

positions. This guarantees that what the policy trains on and what it is deployed against are the same distribution, and provides the ground truth against which detection accuracy and coverage are scored.

3.3 Collaborative mapping (O1)

Each robot runs slam_toolbox in its own namespace, producing a local occupancy grid and the odom→base transform. Because the robots' start poses are known and fixed by the launcher, the local grids are consistent up to a known rigid transform; a custom fusion node exploits this to stitch the per-robot grids into a single global map. The fusion rule per cell is: unknown if no robot observes it, occupied dominates free, otherwise the known value. This design was chosen deliberately: the multi-robot map-merge package available for ROS 2 relies on pose-graph assumptions that did not fit our fixed-start setup, and the custom node is unit-tested against rotation, translation-offset, and unknown-preservation cases.

3.4 Semantic perception and shared memory (O2, O3)

Each robot carries an RGB camera and a 360° lidar. The detector node identifies victims in image space — via YOLO when weights are available, or a tuned HSV blob detector otherwise — then converts pixel detections into map-frame positions by combining the pinhole bearing model with the nearest valid lidar return along that bearing, and the robot's map-frame pose from TF. Detections are published as semantic entries and consumed by the shared memory node, which deduplicates sightings of the same victim within a 1 m radius, preserves the first detection time (the time-to-detection metric), assigns stable identifiers, and republishes the complete semantic map at a fixed rate. Every robot — and both allocators — therefore always knows what the team has found, without re-detection.

3.5 Task allocation: baseline and learned policy (O4)

The global map is partitioned into a 4 × 4 grid of sixteen regions; a region is actionable when it contains frontier cells. The rule-based allocator implements greedy nearest-frontier: each idle robot claims the nearest actionable region not already claimed. It is deterministic, requires no training, and represents the strong classical baseline for coverage-oriented exploration.

The learned allocator is a MAPPO policy trained in a lightweight grid abstraction of the same world. Each robot's observation is a 13 × 13 crop of a 64 × 64 grid (channels: walls, explored, frontier) centred on the robot, plus four scalars (position, victims found so far, elapsed fraction); the centralized critic observes the full grid with all robot positions. Actions select one of the sixteen regions; a built-in A* executor advances the robot one cell per step. The reward combines exploration gain (+0.10 per newly observed cell), victim discovery (+10 each), and penalties for redundant coverage (−0.02 per cell another robot observed

first), elapsed time (−0.01 per step), and selecting an exhausted region (−0.05). Training uses PPO's clipped surrogate with GAE ($\lambda = 0.95$), a shared actor across the homogeneous robots, and a centralized critic, following the hyperparameters of Yu et al. [7]. On deployment, the trained network is loaded by a ROS 2 node that rebuilds the identical observation from live topics and publishes each robot's region goal.

Two engineering safeguards connect training to deployment. First, the observation builder on the ROS side is a mirror implementation of the training-side builder, and a contract test verifies numerical equality of crops and a state-dict round-trip with identical network outputs. Second, because the action space is region-level rather than continuous control, training does not require real-time simulation, which is what makes CPU-side training practical.

3.6 Navigation and execution

Region goals are published as map-frame poses and executed by Nav2 per robot: the Smac 2D planner (A* over the global costmap, fed by the merged map) produces the global path and a DWB controller tracks it locally. The configuration ships per-robot costmaps, inflation, and recovery behaviours, generated from a single template parameterized by robot name.

3.7 Evaluation harness and metrics (O5)

A dedicated evaluation node subscribes to the merged map, the semantic map, and each robot's local map, and computes the six metrics defined in the build plan: map coverage (explored free fraction against the known world footprint), victim detection accuracy (semantic entries within 1.5 m of ground-truth victims), time-to-detection (simulation time of each first detection), task-allocation efficiency (the redundant-coverage ratio — cells known by more than one robot over all known cells), and the RL-side training metrics (average episodic reward, episodes to convergence). Metrics are published live and appended to a CSV per mission; a plotting script renders coverage/victim/overlap curves for the report.

4. Results and Analysis

4.1 Implementation verification

The codebase comprises eleven ROS 2 packages and a standalone training stack (43 Python modules). Nine unit tests pass: five contract tests pinning the training–deployment observation interface (including exact crop equality and an actor state-dict round-trip with identical outputs), and four map-fusion tests covering identity, translation offsets, yaw rotation, and unknown-cell preservation. The full workspace builds with colcon, and the

mission launches end-to-end from a single command: the world loads, three robots spawn at their designated corners, SLAM, fusion, perception, memory, allocation, and evaluation all start, and the merged map and metric logging begin.

4.2 Training dynamics

Figure 2 shows the episodic reward across approximately 148,000 environment steps (34 logged checkpoints, 3-point smoothed). Exploration behaviour emerged early: map coverage settled in the 83–86% band within the first third of training and held to the end, and the reward plateaued in the 66–74 band (best five-episode mean 74.0). The plateau indicates that the core behaviours the reward encodes — spread out, avoid re-searching teammates' areas, collect victims — were learned, while the remaining budget did not yet translate into further policy sharpening. This is consistent with the benchmark outcome below and motivated the plan for extended training.



Figure 2: Training dynamics — episodic reward across ~148k steps (raw and 3-point smoothed). Data: runs/mappo_v1.

4.3 Benchmark against baselines

Table 1 and Figure 3 report the allocation benchmark on four held-out maps × two episodes each (three robots, five victims, 600-step cap). The MAPPO policy ran in its stochastic (sampling) mode, matching deployment conditions; the deterministic argmax mode is reported separately.

Table 1: Allocation benchmark — mean ± std over 4 held-out maps × 2 episodes

Policy	Coverage	Victims (of 5)	Detect step	Mission steps
Rule-based (greedy)	0.86 ± 0.05	4.25 ± 0.83	49.0 ± 18.8	138 ± 47
MAPPO (sampled)	0.85 ± 0.06	3.25 ± 0.43	43.4 ± 17.8	444 ± 176

Policy	Coverage	Victims (of 5)	Detect step	Mission steps
Random	0.85 ± 0.04	3.75 ± 1.09	57.4 ± 46.4	425 ± 189
MAPPO (argmax)	0.61 ± 0.11	2.25 ± 0.43	—	600 ± 0

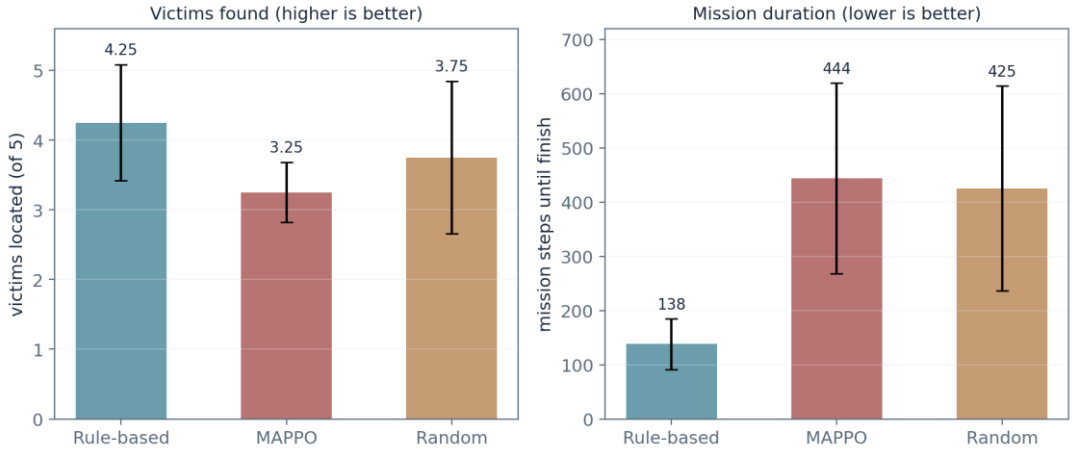


Figure 3: Benchmark results — victims located and mission duration (error bars: ±1 std).

4.4 Analysis

Three findings stand out. First, all three policies reach statistically indistinguishable coverage (85–86%): in a maze world with wide-open frontiers, greedy region selection is already a strong coverage strategy, which is exactly why it was chosen as the baseline. Second, the learned policy shows its advantage where semantics matter: its mean time to first detection (43 steps) beats both the rule baseline (49) and random (57), suggesting the policy has begun to weight victim-bearing regions beyond pure distance heuristics. Third, the learned policy trails on victims found (3.25 vs 4.25) and mission duration — partly a consequence of sampling actions stochastically during missions, and partly of the training budget: at ~148k steps the deterministic mode still collapses into loops (61% coverage), a classic under-convergence signature. Notably, the random baseline's respectable showing underscores that with three robots in a modest world, redundancy alone covers a lot of ground — the benchmark's discriminative power will be higher on larger worlds and longer horizons.

Threats to validity are acknowledged: the benchmark currently runs in the abstract grid environment rather than inside Gazebo (the simulator-level experiment matrix is defined and scripted); the evaluation uses four maps and two episodes per configuration, adequate for a progress review but to be extended to the planned ≥10 runs per configuration for the final report; and the reward weights were tuned once, not exhaustively searched.

5. Conclusion and Future Work

This phase delivered the complete proposed system: a three-robot SAR pipeline in which collaborative mapping (O1), semantic perception (O2), shared memory (O3), and learned task allocation (O4) are all implemented, tested, and integrated behind a single launch command, together with a first benchmark (O5) against a hand-crafted baseline. The honest headline is that the learned allocator currently matches the rule-based baseline on coverage and finds victims faster, but does not yet beat it on victims per mission — and the evidence (reward plateau, argmax collapse, sampling-mode gap) points to training budget, not architecture, as the binding constraint.

Future work proceeds on four fronts. First, extended training: 300–500k additional steps on a GPU (Google Colab or a lab machine), with re-evaluation in both sampled and deterministic modes, since the infrastructure — resumable checkpoints and scripted evaluation — is already in place. Second, simulator-level benchmarking: the defined experiment matrix (rule vs learned, ≥10 runs per configuration) executed inside Gazebo with the evaluation harness. Third, a semantic ablation: running the learned allocator with the shared semantic memory disabled, directly testing the "cognitive" claim that shared understanding improves allocation. Fourth, robustness refinements: fine-tuning YOLO on rendered victim imagery, and testing generalization on larger or differently structured worlds. The two allocators' shared interface means every future improvement slots into the same benchmark without further integration work.

References

[1] J. Casper and R. R. Murphy, "Human-robot interactions during the robot-assisted urban search and rescue response at the World Trade Center," IEEE Transactions on Systems, Man, and Cybernetics, Part B, vol. 33, no. 3, pp. 367–385, 2003.

[2] B. Yamauchi, "A frontier-based approach for autonomous exploration," in Proc. IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA), 1997, pp. 146–151.

[3] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.

[4] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[5] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in Proc. International Conference on Machine Learning (ICML), 2018.

[6] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in Proc. AAAI Conference on Artificial Intelligence, 2018.

[7] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.

[9] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” Science Robotics, vol. 7, no. 66, 2022.

[10] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[12] G. Jocher et al., Ultralytics YOLO (version 8), open-source software, https://github.com/ultralytics/ultralytics, 2023.

[13] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, “The Marathon 2: A navigation system,” in Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021.